

Word Frequency (OOPS)

```
import re

class WordFrequencyCounter:

    def preprocess_text(self, text: str) -> list:

        text = text.lower()

        text = re.sub(r'^\w\s|$', '', text)

        words = text.split()

        return words

    def compute_word_frequency(self, words: list) -> dict:

        frequency = {}

        for word in words:

            frequency[word] = frequency.get(word, 0) + 1

        return frequency

    def get_most_frequent_word(self, freq_dict: dict) -> tuple:

        if not freq_dict:
            return None

        result = max(
            freq_dict.items(),
            key=lambda x: x[1]
        )

        return result

    def filter_words_by_frequency(self, freq_dict: dict, n: int) -> dict:

        result = {}

        for word, count in freq_dict.items():

            if count >= n:
                result[word] = count

        return result
```

Employee Attendance Analyzer (Pandas)

```
import pandas as pd

class AttendanceAnalyzer:

    def create_attendance_df(self, data: list) -> pd.DataFrame:

        df = pd.DataFrame(data, columns=["EmployeeID", "Department", "Date", "Attendance"])

        return df

    def compute_monthly_attendance_rate(self, df: pd.DataFrame) -> pd.DataFrame:

        df = df.copy()

        # Extract YYYY-MM from the Date string
        df["Month"] = df["Date"].str[:7]

        # Count total attendance records
        total = (df.groupby(["EmployeeID", "Month"]).size().reset_index(name="Total"))
```

```

# Keep only Present records
present_df = df[df["Attendance"] == "Present"]

# Count Present records
present = (present_df.groupby(["EmployeeID", "Month"]).size().reset_index(name="Present"))

# Combine total and present counts
result = total.merge(present,on=["EmployeeID", "Month"],how="left")

# Employees with no Present records get 0
result["Present"] = result["Present"].fillna(0)

# Calculate percentage
result["Attendance Rate"] = (result["Present"] / result["Total"] * 100)

return result[["EmployeeID", "Month", "Attendance Rate"]]

def add_absence_flag(self,df: pd.DataFrame) -> pd.DataFrame:

    df = df.copy()

    df["IsAbsent"] = (df["Attendance"] == "Absent").astype(int)

    return df

def high_absentees(self,df: pd.DataFrame,threshold: int) -> pd.DataFrame:

    absent = df[df["Attendance"] == "Absent"]

    result = (absent.groupby("EmployeeID").size().reset_index(name="Absence Count"))

    result = result[result["Absence Count"] > threshold]

    return result

def department_attendance_summary(self,df: pd.DataFrame) -> pd.DataFrame:

    result = pd.crosstab(df["Department"],df["Attendance"])

    result = result.reindex(columns=["Present", "Absent", "Leave"],fill_value=0)

    result = result.reset_index()

    result.columns.name = None

    return result

```

Start coding or [generate](#) with AI.